

Java EE — Module 3

Java Server Pages (JSP)

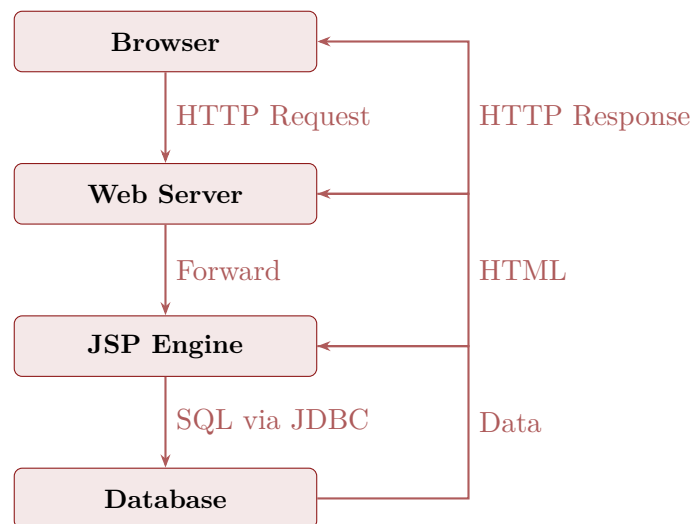
Detailed Study Notes

Q1. What is JSP? Explain its advantages.

JSP (JavaServer Pages) is a server-side technology that allows developers to create **dynamic web pages** by embedding Java code directly within HTML. A JSP file has the extension `.jsp` and is processed on the server before the resulting HTML is sent to the client's browser.

JSP is part of Java EE and runs inside a **web container** (e.g., Apache Tomcat). Internally, every JSP page is automatically converted into a Servlet by the container — but JSP is far easier to write because it lets you focus on the HTML structure with Java logic inserted only where needed.

How JSP fits in a web application:



Advantages of JSP:

- **Separation of concerns** — HTML designers write the page layout; Java developers embed only the logic needed, keeping the code cleaner than pure Servlets.
- **Easy to write** — HTML-like syntax with `<% %>` tags is more intuitive than writing `out.println("<html>")` in a Servlet.
- **Auto-compiled** — The container automatically translates and compiles the JSP into a Servlet; no manual compilation step needed.
- **Reusability** — JSP supports JavaBeans, custom tag libraries, and JSTL, all of which promote code reuse.
- **Implicit objects** — JSP provides built-in objects (`request`, `response`, `session`, etc.) without any declaration.

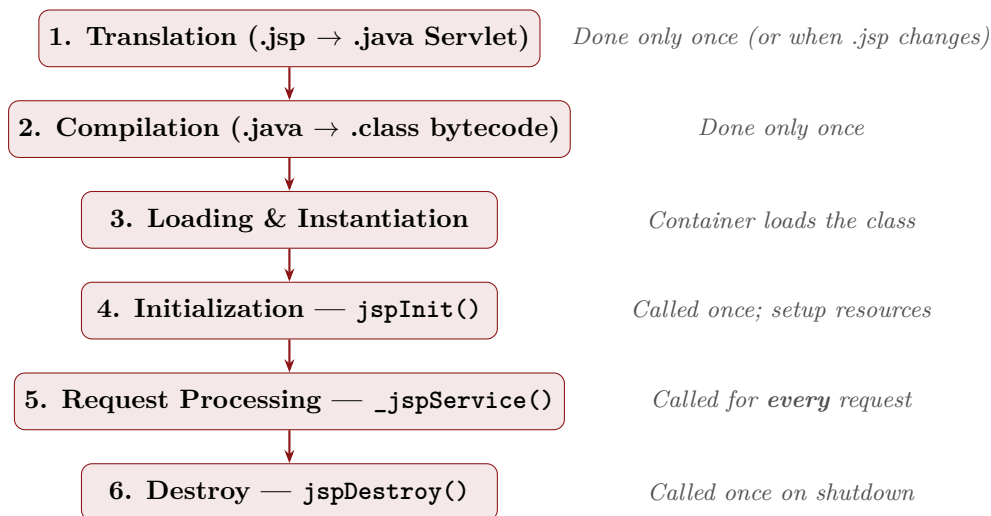
- **Platform independent** — Runs on any web container that supports the JSP specification (Tomcat, JBoss, GlassFish).
- **Integration with Java EE** — Naturally integrates with Servlets, JDBC, EJB, and other Java EE technologies.
- **Expression Language (EL)** — Reduces Java code in JSP pages, making them cleaner and more maintainable.

JSP vs Servlet: A Servlet embeds HTML inside Java (`out.println(...)`). JSP embeds Java inside HTML. For *presentation*, JSP is clearly simpler. Both compile to the same bytecode — JSP is just a convenient wrapper.

Q2. Explain the JSP lifecycle.

The **JSP lifecycle** describes the sequence of steps that a JSP page goes through from the moment it is first requested until it is destroyed. It is managed by the **web container** (e.g., Tomcat). A JSP page is ultimately converted into a Servlet and follows the Servlet lifecycle, but with extra translation steps.

Lifecycle phases:



Step-by-step explanation:

1. **Translation:** When a JSP page is first requested (or has been modified), the web container translates it into a Java Servlet source file (.java). This happens *only once* unless the JSP changes.
2. **Compilation:** The generated .java file is compiled into a .class bytecode file by the Java compiler. Again, done once per version of the JSP.
3. **Loading & Instantiation:** The container loads the compiled class and creates an instance (object) of it — just like any Servlet.
4. **Initialization — jspInit():** The container calls jspInit() once. You can override this to set up resources like database connections.

5. **Request Processing — `_jspService()`**: Called for **every incoming request**. This is the core method — it handles the HTTP request and generates the response. You cannot override this method; it is auto-generated.
6. **Destroy — `jspDestroy()`**: Called once when the container removes the JSP (e.g., during server shutdown or undeployment). Use it to release resources.

Key point: Steps 1 and 2 (translation and compilation) are one-time costs. After that, every subsequent request goes directly to `_jspService()`, making JSP execution fast. The translated Servlet stays in memory and serves all concurrent requests.

Q3. What are JSP scriptlet, expression, and declaration?

JSP provides three **scripting elements** to embed Java code inside an HTML page. They differ in *where* the embedded Java code ends up in the generated Servlet.

1. Scriptlet — `<% code %>`

- Contains **executable Java statements**.
- Code is placed inside the `_jspService()` method of the generated Servlet.
- Can use variables, loops, conditions, and call methods.
- Can access all implicit objects (`request`, `response`, etc.).

```
<%  
int a = 10, b = 20;  
int sum = a + b;  
out.println("Sum = " + sum);  
%>
```

2. Expression — `<%= expression %>`

- Evaluates a Java expression and **outputs its value** directly into the HTML response.
- No semicolon inside the tag.
- Equivalent to `out.print(expression)`.
- Used for printing values inline within HTML.

```
<p>Today's date is: <%= new java.util.Date() %></p>  
<p>User name: <%= request.getParameter("name") %></p>
```

3. Declaration — `<%! code %>`

- Declares **instance variables or methods** that belong to the generated Servlet class itself (outside `_jspService()`).
- Variables declared here are shared across requests (instance-level scope).
- Methods declared here can be called from scriptlets/expressions.

```

<%!
int counter = 0;  // instance variable (shared across requests)

String greet(String name) {
    return "Hello, " + name + "!";
}
%>
<!-- Use the method in an expression -->
<p><%= greet("Alice") %></p>
<p>Page hits: <%= ++counter %></p>

```

Element	Syntax	Where it goes	Purpose
Scriptlet	<% ... %>	Inside <code>_jspService()</code>	Java statements, logic
Expression	<%= ... %>	Inside <code>_jspService()</code> as <code>out.print()</code>	Output a value to page
Declaration	<%! ... %>	Class level (outside <code>_jspService()</code>)	Declare variables/methods

Modern practice: Scriptlets are considered bad practice in modern JSP because they mix logic and presentation. Use **EL** (`${...}`) and **JSTL** tags instead for cleaner, more maintainable pages.

Q4. What are JSP implicit objects? Explain any five.

Implicit objects are pre-defined Java objects that the JSP container automatically makes available in every JSP page — you can use them directly in scriptlets and expressions **without any declaration or initialization**. There are **9 implicit objects** in total.

Object	Type	Purpose
request	HttpServletRequest	Client's HTTP request data
response	HttpServletResponse	Server's HTTP response
out	JspWriter	Output stream to browser
session	HttpSession	User session data
application	ServletContext	App-wide shared data
config	ServletConfig	Servlet config parameters
pageContext	PageContext	Access to all scopes/objects
page	Object (this)	Reference to current servlet
exception	Throwable	Error info (error pages only)

Explanation of five important implicit objects:

1. request — HttpServletRequest

Represents the HTTP request made by the client. Used to retrieve form data, request parameters, headers, and client information.

```

<%
String name = request.getParameter("username");
String ip   = request.getRemoteAddr();
out.println("Hello " + name + ", your IP is " + ip);

```

```
%>
```

Key methods: `getParameter()`, `getHeader()`, `getMethod()`, `setAttribute()`, `getAttribute()`.

2. response — `HttpServletResponse`

Represents the HTTP response sent back to the client. Used to set content type, redirect the user, add cookies, or set response headers.

```
<%  
response.setContentType("text/html");  
response.sendRedirect("login.jsp"); // redirect browser  
%>
```

Key methods: `sendRedirect()`, `setContentType()`, `addCookie()`, `setHeader()`.

3. out — `JspWriter`

The output stream used to write HTML content to the browser. Similar to `System.out`, but writes to the HTTP response body.

```
<%  
out.println("<h2>Welcome to JSP!</h2>");  
out.print("Current time: " + new java.util.Date());  
%>
```

Key methods: `print()`, `println()`, `flush()`, `close()`.

4. session — `HttpSession`

Represents the user's session. Data stored here persists across multiple requests from the same user (until session expires or is invalidated). Used for login tracking, shopping carts, etc.

```
<%  
// Store data  
session.setAttribute("loggedInUser", "Alice");  
  
// Retrieve data  
String user = (String) session.getAttribute("loggedInUser");  
out.println("Logged in as: " + user);  
  
// Invalidate on logout  
session.invalidate();  
%>
```

Key methods: `setAttribute()`, `getAttribute()`, `invalidate()`, `getId()`, `setMaxInactiveInterval()`.

5. application — `ServletContext`

Represents the entire web application context. Data stored here is shared across *all* users and *all* JSP/Servlet pages in the application. Useful for application-wide counters or configuration.

```
<%  
// Increment a global visit counter
```

```
Integer visits = (Integer) application.getAttribute("visits");
if (visits == null) visits = 0;
application.setAttribute("visits", ++visits);
out.println("Total site visits: " + visits);
%>
```

Key methods: `setAttribute()`, `getAttribute()`, `getInitParameter()`, `log()`.

Scope comparison:

`pageContext` — current page only `request` — single request `session` — one user, multiple requests `application` — all users, entire app

Q5. Explain JSP directives with examples.

JSP Directives provide global information about a JSP page to the web container. They **do not produce any visible output** — they simply instruct the container on how to process the page. Directives are evaluated at **translation time** (when the JSP is converted to a Servlet).

Syntax: `<%@ directive attribute="value" %>`

There are **three types** of JSP directives:

1. Page Directive — `<%@ page ... %>`

Defines page-level properties: content type, language, imports, error handling, session, etc.

```
<%@ page language="java"
contentType="text/html; charset=UTF-8"
import="java.util.*, java.sql.*"
session="true"
errorPage="error.jsp"
isErrorPage="false"
buffer="8kb"
autoFlush="true" %>
```

Key attributes of page directive:

Attribute	Purpose	Default
<code>language</code>	Scripting language (always java)	<code>java</code>
<code>contentType</code>	MIME type of response	<code>text/html</code>
<code>import</code>	Java classes to import	<code>none</code>
<code>session</code>	Whether session is available	<code>true</code>
<code>errorPage</code>	JSP to forward to on exception	<code>none</code>
<code>isErrorPage</code>	Whether this page is an error handler	<code>false</code>
<code>buffer</code>	Output buffer size	<code>8kb</code>
<code>isThreadSafe</code>	Whether JSP handles concurrent requests	<code>true</code>

2. Include Directive — `<%@ include ... %>`

Includes the content of another file **at translation time** — the included file's content is merged into the current JSP before it is compiled. This is a *static* include.

```

<!-- Include a common header on every page -->
<%@ include file="header.jsp" %>

<h1>Main Content Here</h1>

<%@ include file="footer.jsp" %>

```

- File path is relative to the current JSP.
- Changes to the included file require recompilation of the parent JSP.
- Included file can use scriptlets/variables of the parent page (same scope).

3. Taglib Directive — `<%@ taglib ... %>`

Declares a **custom tag library** or JSTL to be used in the JSP page. Must be declared before using any tags from the library.

```

<!-- Declare JSTL Core library -->
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>

<!-- Declare JSTL Functions library -->
<%@ taglib uri="http://java.sun.com/jsp/jstl/functions" prefix="fn" %>

<!-- Use the declared tags -->
<c:out value="${name}" />
<c:if test="${age >= 18}">Adult</c:if>

```

Directive	Purpose	Evaluated at
page	Page-level settings	Translation time
include	Static file inclusion	Translation time (merged before compile)
taglib	Declare custom/JSTL tag library	Translation time

Q6. What are JSP action elements? Explain any three.

JSP Action Elements are XML-based tags that instruct the web container to perform specific actions **at request time** (unlike directives which act at translation time). They use XML syntax and look like: `<jsp:actionName attribute="value" />`.

Action elements allow you to control Servlet behavior, manage JavaBeans, include content, and forward requests — all *without writing Java code*.

The standard JSP action elements include: `jsp:include`, `jsp:forward`, `jsp:useBean`, `jsp:setProperty`, `jsp:getProperty`, `jsp:param`, `jsp:plugin`.

1. `<jsp:include>` — Dynamic Include

Includes the output of another resource (JSP, Servlet, HTML) **at request time**. Unlike the include directive (static, at translation time), this includes the resource dynamically — the

included resource is processed separately and its output is inserted.

```
<!-- Dynamically include a page with optional parameter -->
<jsp:include page="header.jsp">
<jsp:param name="title" value="Home Page" />
</jsp:include>

<h1>Welcome to the site</h1>

<jsp:include page="footer.jsp" />
```

- Changes to the included page are reflected without recompiling the parent.
- The included file runs in its own scope — cannot use parent's variables.
- `<jsp:param>` passes parameters to the included page via `request.getParameter()`.

2. `<jsp:forward>` — Request Forwarding

Forwards the entire request to another resource (JSP, Servlet, HTML). The browser URL does not change (server-side redirect). Processing of the current page stops immediately.

```
<%
String role = (String) session.getAttribute("role");
%>
<% if ("admin".equals(role)) { %>
<jsp:forward page="admin.jsp" />
<% } else { %>
<jsp:forward page="user.jsp">
<jsp:param name="message" value="Welcome back!" />
</jsp:forward>
<% } %>
```

- Equivalent to `RequestDispatcher.forward()` in a Servlet.
- The forwarded page receives the same `request` and `response` objects.
- Any output written to `out` *before* the forward is discarded (buffer is cleared).

3. `<jsp:useBean>`, `<jsp:setProperty>`, `<jsp:getProperty>`

These three work together to use **JavaBean components** in JSP pages without scriptlets.

```
<!-- Create or retrieve a JavaBean -->
<jsp:useBean id="student" class="com.example.Student" scope="session" />

<!-- Set properties on the bean -->
<jsp:setProperty name="student" property="name" value="Alice" />
<jsp:setProperty name="student" property="marks" value="95" />

<!-- Get and display a property -->
<p>Student: <jsp:getProperty name="student" property="name" /></p>
<p>Marks: <jsp:getProperty name="student" property="marks" /></p>
```

- `scope` can be `page`, `request`, `session`, or `application`.

- `<jsp:setProperty name="b" property="*" />` auto-maps all request parameters to matching bean properties.

Q7. Explain how JavaBeans are used in JSP.

A **JavaBean** is a reusable Java class that follows a specific set of conventions:

- It must have a **public no-argument constructor**.
- All fields must be **private**.
- Public **getter and setter** methods must be provided for each field.
- It should implement `java.io.Serializable` (optional but recommended).

JavaBeans separate business data from JSP presentation logic. Instead of storing data in scriptlets, you store it in a bean and access it cleanly from JSP.

Step 1 — Define the JavaBean class:

```
package com.example;

public class Student implements java.io.Serializable {
    private int id;
    private String name;
    private double marks;

    // No-arg constructor (mandatory)
    public Student() {}

    // Getters and Setters
    public int getId()      { return id; }
    public void setId(int id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public double getMarks() { return marks; }
    public void setMarks(double marks) { this.marks = marks; }
}
```

Step 2 — Use the bean in JSP:

```
<%@ page contentType="text/html; charset=UTF-8" %>

<!-- Instantiate or retrieve bean from session scope -->
<jsp:useBean id="student" class="com.example.Student" scope="session" />

<!-- Populate from form parameters automatically -->
<jsp:setProperty name="student" property="*" />

<!-- Or set individually -->
<jsp:setProperty name="student" property="name" value="Alice" />
<jsp:setProperty name="student" property="marks" value="88.5" />
```

```
<html><body>
<h2>Student Details</h2>
<p>Name: <jsp:getProperty name="student" property="name" /></p>
<p>Marks: <jsp:getProperty name="student" property="marks" /></p>
</body></html>
```

Bean scopes:

Scope	Lifetime
page	Only during the current page processing
request	For the duration of the current HTTP request
session	For the user's entire session (across multiple pages)
application	For the entire lifetime of the web application

MVC pattern with JavaBeans: Servlet (Controller) creates and populates a JavaBean, stores it in request/session scope, then forwards to JSP (View). The JSP reads the bean's properties using `<jsp:getProperty>` or EL (`${student.name}`) — no Java code in JSP at all.

Q8. What is JSP Expression Language (EL)? Explain its features.

JSP Expression Language (EL) is a simplified language introduced in JSP 2.0 to **access data stored in beans, maps, lists, and request/session attributes** without writing Java scriptlets. EL expressions use the syntax `${expression}`.

EL makes JSP pages cleaner, more readable, and easier to maintain by eliminating Java code from the view layer.

Basic syntax:

```
<!-- Without EL (using scriptlet) -->
<p>Hello, <%= (String)request.getAttribute("name") %>!</p>

<!-- With EL -->
<p>Hello, ${name}!</p>
```

Features of EL:

1. Accessing scoped attributes:

EL automatically searches `page` → `request` → `session` → `application` scope for a named attribute.

```
${username}           <!-- finds "username" in any scope -->
${sessionScope.user} <!-- looks only in session scope -->
${requestScope.msg}  <!-- looks only in request scope -->
```

2. Accessing bean properties (dot notation):

```

${student.name}    <!-- calls student.getName() -->
${student.marks}   <!-- calls student.getMarks() -->

```

3. Accessing maps and lists (bracket notation):

```

${map["key"]}      <!-- map.get("key") -->
${list[0]}         <!-- list.get(0) -->
${param["username"]} <!-- request.getParameter("username") -->

```

4. Arithmetic, relational, and logical operators:

```

${10 + 5}          <!-- outputs 15 -->
${a > b}           <!-- true or false -->
${a == b ? "yes" : "no"} <!-- ternary -->
${not empty name}  <!-- true if name is not null/empty -->

```

5. EL implicit objects:

EL Implicit Object	Equivalent to
param	request.getParameter()
paramValues	request.getParameterValues()
header	request.getHeader()
cookie	request.getCookies()
sessionScope	session.getAttribute()
requestScope	request.getAttribute()
applicationScope	application.getAttribute()
initParam	context.getInitParameter()

6. empty operator:

```

${empty list}      <!-- true if list is null or has no elements -->
${not empty name}  <!-- true if name is non-null and non-empty -->

```

Why use EL over scriptlets?

Scriptlets mix Java and HTML, making pages hard to read and test. EL is concise, safe (no ClassCastException risk), and works seamlessly with JSTL tags. **EL + JSTL** is the modern, recommended approach to JSP development.

Q9. Explain JSTL core tags.

JSTL (JSP Standard Tag Library) is a collection of custom tags that encapsulate common web application tasks — iteration, conditionals, output, URL handling — allowing you to do them **without Java scriptlets**. The **Core** library is declared as: `<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>`

1. <c:out> — Output a value

Outputs the value of an EL expression. Safer than `${}` because it escapes HTML special

characters by default, preventing XSS.

```
<c:out value="${user.name}" default="Guest" />
<!-- If user.name is null, outputs "Guest" -->
<!-- escapeXml="false" disables HTML escaping -->
<c:out value="${htmlContent}" escapeXml="false" />
```

2. <c:set> — Set a variable

Creates or assigns a value to a scoped variable (or sets a property of a bean/map).

```
<c:set var="greeting" value="Hello, World!" scope="session" />
<c:out value="${greeting}" />

<!-- Set a bean property -->
<c:set target="${student}" property="name" value="Alice" />
```

3. <c:remove> — Remove a variable

Removes a scoped variable from any or a specified scope.

```
<c:remove var="greeting" scope="session" />
<!-- greeting is now null in session scope -->
```

4. <c:if> — Conditional (single branch)

Renders body content only if the test expression is true. There is no <c:else> — use <c:choose> for that.

```
<c:if test="${age >= 18}">
<p>You are an adult.</p>
</c:if>

<c:if test="${not empty sessionScope.user}">
<p>Welcome, ${sessionScope.user.name}!</p>
</c:if>
```

5. <c:choose>, <c:when>, <c:otherwise> — if-else chain

Equivalent to Java's if / else if / else block.

```
<c:choose>
<c:when test="${marks >= 90}">
<p>Grade: A</p>
</c:when>
<c:when test="${marks >= 75}">
<p>Grade: B</p>
</c:when>
<c:when test="${marks >= 60}">
<p>Grade: C</p>
</c:when>
<c:otherwise>
<p>Grade: F (Fail)</p>
</c:otherwise>
```

```
</c:otherwise>
</c:choose>
```

6. <c:forEach> — Iteration

The most frequently used JSTL tag. Iterates over a collection (List, Set, Map, array) or a numeric range.

```
<!-- Iterate over a list of student objects -->
<table>
<c:forEach var="s" items="${studentList}" varStatus="status">
<tr>
<td>${status.count}</td>    <!-- row number (1-based) -->
<td>${s.name}</td>
<td>${s.marks}</td>
</tr>
</c:forEach>
</table>

<!-- Numeric range iteration -->
<c:forEach var="i" begin="1" end="10" step="1">
<p>Row ${i}</p>
</c:forEach>
```

varStatus properties: count (1-based index), index (0-based), first (boolean), last (boolean).

7. <c:forTokens> — Token-based iteration

Splits a string by delimiters and iterates over the tokens.

```
<c:forTokens var="fruit" items="apple,banana,cherry,mango"
delims=",">
<li>${fruit}</li>
</c:forTokens>
<!-- Outputs: apple, banana, cherry, mango as list items -->
```

Additional useful core tags (brief):

- <c:url> — constructs a URL with proper encoding.
- <c:redirect> — redirects the browser to another URL.
- <c:import> — imports content from any URL (like jsp:include but more powerful).
- <c:catch> — catches exceptions without disrupting page flow.

Q10. Explain JSTL function tags.

The **JSTL Functions library** (fn:) provides a set of **string manipulation functions** that can be called from EL expressions in JSP. Since EL by itself has no built-in string functions, the fn: library fills that gap.

Declaration: <%@ taglib uri="http://java.sun.com/jsp/jstl/functions" prefix="fn" %> Usage: inside EL — \${fn:functionName(args)}

Function	Description and Example
fn:length(obj)	Returns the length of a string or collection. Example: <code>\${fn:length("Hello")}</code> → 5
fn:toUpperCase(s)	Converts a string to uppercase. Example: <code>\${fn:toUpperCase("hello")}</code> → HELLO
fn:toLowerCase(s)	Converts a string to lowercase. Example: <code>\${fn:toLowerCase("HELLO")}</code> → hello
fn:substring(s,b,e)	Extracts a substring from index b to e-1. Example: <code>\${fn:substring("Hello",0,3)}</code> → Hel
fn:substringAfter(s,t)	Returns the part of the string after a token. Example: <code>\${fn:substringAfter("a@b","@")}</code> → b
fn:substringBefore(s,t)	Returns the part of the string before a token. Example: <code>\${fn:substringBefore("a@b","@")}</code> → a
fn:trim(s)	Removes leading and trailing spaces. Example: <code>\${fn:trim(" hi ")}</code> → hi
fn:replace(s,f,r)	Replaces occurrences of one string with another. Example: <code>\${fn:replace("a-b","-",".")}</code> → a.b
fn:split(s,d)	Splits a string using a delimiter and returns a String array.
fn:join(arr,sep)	Joins array elements using a separator. Example: <code>\${fn:join(arr,"")}</code>
fn:indexOf(s,sub)	Returns the index of a substring. Example: <code>\${fn:indexOf("Hello","ll")}</code> → 2
fn:contains(s,sub)	Checks whether a string contains a substring. Returns true/false.
fn:containsIgnoreCase(s,sub)	Case-insensitive version of contains(). Returns true/false.
fn:startsWith(s,pre)	Checks whether a string starts with a prefix. Returns true/false.
fn:endsWith(s,suf)	Checks whether a string ends with a suffix. Returns true/false.
fn:escapeXml(s)	Escapes XML/HTML special characters. Example: < → <

Practical example using multiple fn: functions:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/functions" prefix="fn" %>

<c:set var="email" value=" Alice@Example.COM " />

<p>Email (trimmed):    ${fn:trim(email)}</p>
<p>Lowercase:         ${fn:toLowerCase(fn:trim(email))}</p>
<p>Length:            ${fn:length(fn:trim(email))}</p>
<p>Has @:              ${fn:contains(email, "@")}</p>
<p>Username:          ${fn:substringBefore(fn:trim(email), "@")}</p>
<p>Domain:             ${fn:substringAfter(fn:trim(email), "@")}</p>

<!-- Split CSV and loop -->
<c:set var="csv" value="Java,Python,C++,JavaScript" />
<c:forEach var="lang" items="${fn:split(csv, ',')}">
<li>${lang}</li>
</c:forEach>
```

Q11. What is a custom tag library? Explain with example.

A **Custom Tag Library** allows developers to create their own JSP tags that encapsulate complex, reusable logic. Instead of repeating Java code or using scriptlets, you define a tag once and use it like an HTML tag anywhere in your JSP pages.

Custom tags are essentially **Java classes** that implement the tag interface, packaged in a **Tag Library Descriptor (TLD)** file.

Why create custom tags?

- Encapsulate reusable presentation logic.
- Remove Java code from JSP entirely.
- Allow non-Java developers to use tag-based components.
- More reusable than scriptlets.

Components of a custom tag library:

1. **Tag Handler Class** — Java class implementing `SimpleTagSupport` or `TagSupport`.
2. **TLD File (Tag Library Descriptor)** — XML file describing the tag (name, class, attributes).
3. **JSP Page** — Uses the tag after declaring it with `<%@ taglib %>`.

Step 1 — Create the Tag Handler Class:

```
package com.example.tags;

import javax.servlet.jsp.tagext.SimpleTagSupport;
import javax.servlet.jsp.JspWriter;
import java.io.IOException;

public class HelloTag extends SimpleTagSupport {

    private String name; // attribute

    public void setName(String name) {
        this.name = name;
    }

    @Override
    public void doTag() throws IOException {
        JspWriter out = getJspContext().getOut();
        out.println("<h2>Hello, " + name + "! Welcome to our site.</h2>");
    }
}
```

Step 2 — Create the TLD file (WEB-INF/mytags.tld):

```
<?xml version="1.0" encoding="UTF-8" ?>
<taglib xmlns="http://java.sun.com/xml/ns/j2ee" version="2.0">
<tlib-version>1.0</tlib-version>
<short-name>mytags</short-name>
<uri>http://com.example/mytags</uri>

<tag>
<name>hello</name>
<tag-class>com.example.tags.HelloTag</tag-class>
<body-content>empty</body-content>
<attribute>
```

```

<name>name</name>
<required>true</required>
<rtexprvalue>true</rtexprvalue>
</attribute>
</tag>
</taglib>

```

Step 3 — Use the custom tag in JSP:

```

<%@ taglib uri="http://com.example/mytags" prefix="my" %>
<!DOCTYPE html>
<html><body>
  <!-- Use the custom tag like any HTML tag -->
  <my:hello name="Alice" />
  <my:hello name="${sessionScope.username}" />
</body></html>

<!-- Output rendered by doTag(): -->
<!-- <h2>Hello, Alice! Welcome to our site.</h2> -->

```

Custom tag with body content:

```

// Tag that wraps its body in a red box
public class BoxTag extends SimpleTagSupport {
    @Override
    public void doTag() throws IOException, JspException {
        JspWriter out = getJspContext().getOut();
        out.println("<div style='border:2px solid red; padding:10px;'>");
        getJspBody().invoke(null); // render the body content
        out.println("</div>");
    }
}

```

```

<my:box>
<p>This content will be wrapped in a red box.</p>
</my:box>

```

Feature	Scriptlet	Custom Tag
Java in JSP	Yes (messy)	No (clean tag syntax)
Reusability	Low (copy-paste)	High (use tag anywhere)
Non-developer friendly	No	Yes (HTML-like)
Maintenance	Hard	Easy (change handler class)

Modern note: While custom tag libraries are powerful, in practice most teams use **JSTL** + **EL** which covers the majority of use cases. Custom tags are most valuable when you have complex, reusable rendering logic that JSTL cannot handle out of the box.